

UNIVERSIDAD SIGLO 21

Licenciatura en Informática

Modalidad EDH

TRABAJO PRÁCTICO 4

Cómputo paralelo en la nube con MPI y GitHub Codespaces

Materia: Algoritmos Concurrentes y Paralelos

Código Canvas: INF394 - 11805

Profesor titular: Eduardo Enrique Piray

Alumno: Rodríguez, Daniel Sebastián

Legajo: VIN016869

DNI: 43.731.653

Fecha de entrega: 04/05/2026

1. Introducción

El TP4 cierra el cuatrimestre y aplica los conceptos del módulo 4 sobre cómputo paralelo, esta vez usando la nube. La consigna es distinta a los TP's anteriores: el código ya está hecho por el profesor en un repositorio de GitHub (https://github.com/edupiray/tp4_acyp), y lo que hay que hacer es hacer fork del repositorio, abrirlo en GitHub Codespaces, correr los tres programas que vienen adentro (multiplicación de matrices, calculo de pi por el método de Monte Carlo y K-means para agrupar puntos), mirar las salidas y responder preguntas sobre cómo funciona cada uno.

La idea no es escribir código nuevo sino entender qué hace cada parte del que ya está y, sobre todo, por qué se usa una función de MPI en cada lugar. MPI es el estándar para que varios procesos se pasen mensajes entre si, porque a diferencia de los hilos del TP2, los procesos no comparten memoria y necesitan comunicarse explícitamente.

2. Del fork al Codespace

Los pasos que seguí son los que indica la consigna y el README del repositorio:

1. Inicié sesión en mi cuenta de GitHub.
2. Entre al repositorio https://github.com/edupiray/tp4_acyp y toqué el botón Fork (arriba a la derecha). GitHub copia el repositorio en mi cuenta.
3. En mi fork, toqué el botón verde Code, fui a la pestaña Codespaces y al botón + para crear uno nuevo en la rama main. Después de menos de un minuto se abre Visual Studio Code dentro del navegador, con una terminal abajo y todo listo para compilar (Ubuntu 22.04, MPICH instalado, gcc 11.4.0).
4. En la terminal navegué a cada carpeta (ejemplos_base y desafios_ml) y corrí los comandos que figuran en cada README.md.

Lo que arma el entorno es el archivo .devcontainer/devcontainer.json del repositorio:

```
{
  "image": "ubuntu:22.04",
  "postCreateCommand": "apt update && apt install -y mpich libmpich-dev",
  "customizations": {
    "vscode": { "extensions": ["ms-vscode.cpptools"] }
  }
}
```

Esa configuración alcanza para tener todo lo necesario sin instalar nada en la PC propia, que es una de las gracias de Codespaces. El repositorio trae tres carpetas: ejemplos_base con la multiplicación de matrices y desafios_ml con los dos ejercicios de machine learning (Monte Carlo y K-means).

3. Producto de matrices

El programa multiplica dos matrices de 4 x 4 con números enteros aleatorios entre 0 y 9 (la constante N esta fijada en 4 dentro del código). La idea, traducida a pasos: el proceso 0 arma A y B, después se reparten las filas de A entre los procesos con MPI_Scatter (una fila a cada uno, porque hay 4 filas y se corre con 4 procesos), se envia la matriz B entera a todos con MPI_Bcast, cada proceso multiplica su fila por B y obtiene una fila de la matriz resultado C, y al final MPI_Gather junta todas esas filas en el proceso 0 para armar la matriz C completa.

En la terminal del Codespace corrí los dos comandos que figuran en el README.md de ejemplos_base:

```
# Versión C
$ mpicc matmul.c -o matmul_c && mpirun -np 4 ./matmul_c
```

```
# Versión C++
$ mpic++ matmul.cpp -o matmul_cpp && mpirun -np 4 ./matmul_cpp
```

Salida observada (recortada) para la versión C, np=4:

```
Matriz A generada por proceso 0:
7 4 9 2
7 9 5 6
2 6 0 7
5 9 4 6
Matriz B generada por proceso 0:
1 7 5 3
3 2 9 5
9 2 3 3
1 2 6 0
Proceso 0 - Fila recibida de A: 7 4 9 2
Proceso 1 - Fila recibida de A: 7 9 5 6
Proceso 2 - Fila recibida de A: 2 6 0 7
Proceso 3 - Fila recibida de A: 5 9 4 6
Proceso 0 - Resultado parcial: 102 79 110 68
Proceso 1 - Resultado parcial: 85 89 167 81
Proceso 2 - Resultado parcial: 27 40 106 36
Proceso 3 - Resultado parcial: 74 73 154 72
Matriz resultado C:
102 79 110 68
85 89 167 81
27 40 106 36
74 73 154 72
```

Cada proceso muestra la fila que le tocó de A y la fila que calculó para C. Al final el proceso 0 imprime la matriz resultado completa. La versión en C++ hace exactamente lo mismo (cambia la extensión del archivo y se compila con mpic++ en lugar de mpicc) y devuelve otra matriz C distinta, pero solo porque las matrices A y B se generan con números aleatorios distintos cada vez (la semilla cambia con time(NULL)).

4. Monte Carlo para estimar pi

Este programa estima pi con el método de los dardos. La idea es imaginar un cuadrado de lado 1 con un cuarto de círculo de radio 1 adentro, tirar 1.000.000 de dardos al azar dentro del cuadrado y contar cuantos cayeron dentro del cuarto de círculo. Como el área del cuarto de círculo es $\pi/4$ y el área del cuadrado es 1, la proporción entre aciertos y total, multiplicada por 4, da una estimación de pi.

La paralelización es bastante natural: el proceso 0 le avisa a todos el total de dardos con MPI_Bcast, cada proceso tira su parte (por ejemplo 250.000 dardos cada uno si son 4 procesos) usando una semilla diferente time(NULL) + rank para que los números aleatorios sean distintos en cada uno, y al final MPI_Reduce suma todos los aciertos y los manda al proceso 0, que es el que calcula pi y lo imprime.

Salidas observadas para distintos números de procesos:

```
$ mpirun -np 1 ./montecarlo_c
Aciertos totales: 785787 pi estimado: 3.143148 Error: 0.001555

$ mpirun -np 2 ./montecarlo_c
Proceso 0: 392908 Proceso 1: 392989
Aciertos totales: 785897 pi estimado: 3.143588 Error: 0.001995

$ mpirun -np 4 ./montecarlo_c
Procesos 0..3: ~196.500 aciertos cada uno
Aciertos totales: 785729 pi estimado: 3.142916 Error: 0.001323

$ mpirun -np 8 ./montecarlo_c
Procesos 0..7: ~98.200 aciertos cada uno
Aciertos totales: 785807 pi estimado: 3.143228 Error: 0.001635
```

Lo primero que se ve es que cada proceso saca aproximadamente la misma cantidad de aciertos (porque le toca la misma cantidad de dardos), y que el total se mantiene siempre cerca de 785.000, que es justamente lo que se espera porque $\pi/4$ es alrededor del 78,5 % del área del cuadrado. El valor estimado de π queda en torno a 3,142 en todas las corridas, pero NO es idéntico entre una y otra porque cada proceso usa una semilla distinta y los dardos caen en lugares diferentes cada vez.

5. K-means clustering

El K-means es un algoritmo para agrupar puntos en grupos (clusters). Pensado como ejemplo fácil: imaginar 1000 puntos sobre un mapa que se quieren agrupar en 3 barrios. El algoritmo elige tres centros al azar, asigna cada punto al centro más cercano, después recalcula cada centro como el promedio de los puntos de su barrio, y repite el proceso hasta que los centros dejan de moverse mucho (en el código del profe se fijan 20 iteraciones).

La paralelización va así: el proceso 0 arma los puntos y elige los tres centros iniciales al azar, los manda a todos con MPI_Bcast y reparte los puntos con MPI_Scatter (250 puntos para cada proceso si se corre con 4). En cada iteración, cada proceso ve a qué centro queda más cerca cada uno de sus puntos y va sumando las coordenadas por cluster. Después MPI_Allreduce suma esas cuentas y esas sumas de todos los procesos y deja el resultado en TODOS (no solo en el proceso 0), el proceso 0 calcula los nuevos centros y los redistribuye con MPI_Bcast para que todos los usen en la siguiente vuelta.

Salida observada para np=4:

```
$ mpicc kmeans_mpi.c -o kmeans_c -lm
$ mpirun -np 4 ./kmeans_c
=== K-MEANS PARALELO CON MPI ===
Centros iniciales:
C0: (5.16, 6.97)
C1: (8.76, 2.92)
C2: (2.19, 1.69)
Centros finales después de 20 iteraciones:
Cluster 0: (4.28, 7.97) - 350 puntos
Cluster 1: (8.04, 3.40) - 323 puntos
Cluster 2: (2.55, 2.70) - 327 puntos
```

Se ve como los centros se movieron desde la posición aleatoria del principio hasta el centro real de cada grupo, y los 1000 puntos quedaron repartidos bastante parejo entre los tres clusters (350, 323 y 327). Con 2 procesos o con 5 procesos pasa lo mismo, lo único que cambia son los puntos generados al inicio porque la semilla srand(time(NULL)) tira números distintos cada vez.

6. Respuestas a las preguntas del punto 7

Producto de matrices

- **Qué función cumple MPI_Scatter en el código?** Reparte la matriz A entre los procesos: a cada proceso le entrega una fila distinta para que la multiplique.
- **Por qué se usa MPI_Bcast para la matriz B?** Porque cada proceso necesita la matriz B entera para multiplicar su fila de A contra todas las columnas de B, así que se le copia la misma B a todos.
- **Cuántas filas calcula cada proceso cuando se usan 4 procesos para una matriz 4x4?** Una fila cada uno, porque hay 4 filas y 4 procesos (4 dividido 4 es 1).
- **Qué ocurre si se ejecuta con más procesos que filas de la matriz?** Falla: con más procesos que filas el reparto queda mal (cada proceso recibe menos de una fila), aparece basura en los datos y MPI termina con error (lo probé con 8 procesos y la matriz de 4 filas).

- **Por qué solo el proceso 0 imprime el resultado final?** Porque MPI_Gather le manda todas las filas de C únicamente al proceso 0, así que es el único que tiene la matriz completa para imprimir.

Monte Carlo pi

- **Por qué cada proceso necesita una semilla aleatoria diferente?** Porque si todos usan la misma semilla generan exactamente los mismos dardos en los mismos lugares, así no se amplía la muestra. Por eso se usa time(NULL) + rank, para que cada proceso tire números distintos.

- **Qué función MPI permite sumar los aciertos de todos los procesos?** MPI_Reduce con la operación MPI_SUM. Suma los aciertos de cada proceso y deja el total en el proceso 0, que es el único que necesita ese número para calcular pi.

- **Si ejecutas con 1 y con 4 procesos, el valor de pi es exactamente igual? Por qué?** No, da parecido pero no idéntico. En mi corrida me dio 3,143148 con 1 proceso y 3,142916 con 4. La diferencia viene de que con más procesos hay más semillas y los dardos caen en lugares distintos.

- **Cómo afecta el número de procesos al tiempo de ejecución?** Con más procesos el tiempo debería bajar, pero en mi prueba con un millón de dardos en una sola máquina la diferencia es mínima porque MPI tarda más en coordinar lo que ahorra en cómputo (1 y 4 procesos dieron tiempos casi iguales).

- **Podrías modificar el programa para que cada proceso lance distinta cantidad de dardos?** Sí, se podría reemplazar la división pareja (total/size) por un arreglo con la cuota de cada proceso, o usar MPI_Scatterv. El MPI_Reduce final suma igual sin importar cuantos dardos tiro cada uno.

K-means

- **Qué diferencia hay entre MPI_Reduce y MPI_Allreduce? Por qué aquí usamos MPI_Allreduce?** MPI_Reduce deja el resultado solo en el proceso 0; MPI_Allreduce lo deja en todos. Aca lo necesitamos en todos porque cada proceso tiene que saber los nuevos centros para reasignar sus puntos en la siguiente vuelta.

- **Si el número de procesos no divide exactamente a N_PUNTOS, cómo podrías distribuir los datos?** Cambiando MPI_Scatter por MPI_Scatterv, que permite mandarle a cada proceso una cantidad distinta de puntos en lugar de partes iguales.

- **Por qué los centros iniciales se eligen aleatoriamente? Qué riesgo existe?** Porque empezar al azar es fácil y no requiere conocer los datos. El riesgo es que la elección sea mala (por ejemplo dos centros muy cerca) y el resultado quede atrapado en una solución poco útil.

- **Qué papel cumple MPI_Bcast dentro del bucle principal?** Reparte los nuevos centros (que calcula el proceso 0) a todos los demás, para que en la siguiente vuelta del ciclo todos los procesos trabajen con los mismos centros.

- **Al aumentar MAX_ITER o la cantidad de puntos, cómo se comporta la relación cómputo/comunicación?** Al aumentar las iteraciones o los puntos, los procesos pasan más tiempo calculando y proporcionalmente menos tiempo comunicándose, con lo cual la paralelización se vuelve más eficiente.

7. Conclusión

Los tres ejercicios muestran tres maneras distintas de paralelizar con MPI. La multiplicación de matrices es la más típica: el proceso 0 reparte el trabajo, todos calculan su parte y al final se junta el resultado en uno solo. Monte Carlo es más simple porque cada proceso trabaja por su cuenta sin necesitar nada de los demás, y solo al final se suman los resultados. K-means es el

más interesante porque mezcla los dos enfoques: cada proceso trabaja con sus puntos pero al final de cada vuelta TODOS necesitan saber los nuevos centros para seguir, y ahí aparece MPI_Allreduce.

Lo bueno de hacer el TP en GitHub Codespaces es que no hubo que instalar MPI ni configurar nada en mi PC, el devcontainer.json deja todo listo en un minuto. Eso permite enfocarse en entender por que cada función de MPI esta donde esta, en lugar de pelear con la instalacion. Las preguntas del punto 7 ayudan justamente a eso, a fijarse por qué en un caso se usa Reduce y en otro Allreduce, por que se reparte una matriz y la otra se replica, etc.

Referencias

Piray, E. (2026). Repositorio tp4_acyp. https://github.com/edupiray/tp4_acyp

Universidad Siglo 21. (2026). Algoritmos Concurrentes y Paralelos: Lecturas del Módulo 4. Material de cátedra (cursada Marzo-Mayo 2026).

MPICH Project. (2024). MPICH user documentation. <https://www.mpich.org/documentation/>